

Project JARVIS

A personal, **AI-driven career & life operating system**. This document is the end-to-end, build-ready technical spec — data sources, agents, models, MCP tools, databases, keys, and security — for a private “Operator” system and a public “Stage” demo.

OWNER

Solo builder + AI assistants

FIRST MODULE

Job Application Package

CORE STACK

Python · FastAPI · LangGraph

INTERFACE

Telegram + WhatsApp

BUDGET TARGET

≈ \$0–20 / month

00 · EXECUTIVE SUMMARY

What we are building & the rules of the build

Jarvis is a set of cooperating AI agents that run one person's career and life like an operating system. It searches and applies to jobs, warms a professional network, invents and pitches startup ideas, handles a WhatsApp rides bot, chases freelance leads, and narrates the whole thing back to you. It has two deployments: a **Private "Operator"** instance on your real accounts, and a **Public "Stage"** demo on your portfolio that mirrors the system with synthetic data.

This spec turns the product idea into something a solo builder can actually ship, cheaply, with AI coding assistants. Every section answers a concrete build question: where the data comes from, which models and agents do the work, what tools (MCP) they call, what tables store the state, which keys and accounts you need, and how it stays safe and legal.

Cheap by default

Free tiers, local open-source models for bulk work, paid models only for the last 5% of quality. Target $\approx$ \$0–20/mo.

AI-driven, human-gated

Agents do the thinking and drafting. Any action that touches the outside world passes an approval gate until you trust it.

PRINCIPLE 03

ToS-safe over clever

Prefer official APIs and public JSON feeds. Treat LinkedIn scraping as a last resort with real account-ban risk.

PRINCIPLE 04

Ship one module first

The Job Application Package is the MVP. Everything else plugs into the same agent + memory + database core.

00·A · WORKING AGREEMENT

You build it — I coach. Teaching-first.

The point of this project isn't just to *have* Jarvis — it's for **you to learn to build it**. So we work as a pair: you write the code, I explain, review, unblock, and hand you the next small piece. I don't build the whole thing for you. Every phase in the roadmap doubles as a lesson.

YOUR ROLE

- Write the code, run it, break it, fix it.
- Make the decisions; own the accounts & keys.
- Ask "why" freely — no question is too basic.
- Learn one concept per step, not all at once.

MY ROLE (COACH)

- Explain concepts before code, in plain terms.
- Hand you small, testable tasks — not giant dumps.
- Review your code, debug with you, suggest fixes.
- Give scaffolds/snippets to learn from, not to blindly paste.

Skills you'll pick up, in order

We only learn each when a build step needs it — just-in-time, never a textbook up front.

Phase 0 Python + async basics, virtualenv/uv, reading APIs, env vars & secrets, a first Telegram bot.

Phase 1 Calling LLMs, prompt design, LangGraph nodes/state, Postgres + SQL, embeddings & dedup.

Phase 2-3	MCP tools, Playwright + browser-use, vector memory, guardrails & approval gates.
Phase 4-5	Deployment, observability/tracing, Next.js for the public demo, security hardening.

HOW EACH STEP RUNS

1) I explain the concept & why → 2) I give you a small task + a hint or skeleton → 3) you build and run it → 4) we review together and fix → 5) we document it and move on. Tell me your current comfort level with Python and we'll set the starting difficulty.

01 · SYSTEM ARCHITECTURE

Two faces, one core, five layers

Both the private operator and the public demo run the *same* code with different config: real tools + real data vs. mock tools + synthetic data. Everything sits on a shared five-layer spine.

LAYERED ARCHITECTURE (TOP → BOTTOM)	
1 · Interface	Telegram bot (primary), WhatsApp (rides + freelance), the public portfolio web UI. Where you talk to Jarvis and approve actions.
2 · Orchestration	FastAPI (webhooks + REST) and LangGraph (agent graphs, routing, state machines). A scheduler fires the daily runs. The "brain stem."
3 · Agents	Specialist agents (Job Scout, Fit Scorer, Resume Tailor, Outreach Writer, Startup Scout, Rides Watcher...) coordinated by a Supervisor.
4 · Tools (MCP)	Model Context Protocol servers expose capabilities: job feeds, browser, Postgres, Gmail/Calendar, filesystem, web search. Agents call tools; tools do the world.
5 · Data & Memory	Postgres (state) + pgvector (semantic memory) + object storage (resumes/PDFs) + Redis (queues/cache). The long-term record of everything Jarvis knows and did.

PRIVATE • OPERATOR MODE

- Runs on real Gmail, Calendar, job boards, WhatsApp.
- Entry via Telegram + WhatsApp.
- Real tools, real memory, approval gates.
- Deployed privately (home server / private cloud).

PUBLIC • STAGE MODE

- Synthetic personas + mock tools only.
- Lives on the portfolio site.
- Narrates what the real system would do.
- Rate-limited, PII-redacted, sandboxed.

02 • TECHNOLOGY STACK

Concrete choices, all free or near-free

Chosen for a solo Python builder using AI coding assistants. Everything here has a generous free tier or is open-source and self-hostable.

CONCERN	CHOICE	WHY / COST
Language	Python 3.12	Best agent + scraping ecosystem. Free.
API / webhooks	FastAPI + Uvicorn	Async, tiny, handles Telegram/WhatsApp webhooks. Free.
Agent framework	LangGraph	Graph/state-machine agents, durable, model-agnostic. Free OSS.
Tool protocol	MCP (Model Context Protocol)	Standard way to expose tools to any model. Free OSS servers.
Database	Postgres + pgvector (Supabase)	State + semantic memory in one. Free tier (500MB).
Queue / cache	Redis (Upstash) + APScheduler	Background jobs + daily cron. Free tiers.
File / blob storage	Supabase Storage	Resume PDFs, generated docs. Free tier.
Browser automation	Playwright + browser-use	AI-driven form filling for auto-apply. Free OSS.
Local model runtime	Ollama	Runs Llama/Qwen locally for bulk work. Free.
Model gateway	OpenRouter + LiteLLM	One API for many models incl. free ones. Pay-per-use.
Interface	Telegram Bot API · Baileys (WhatsApp)	Free. Baileys = existing rides bot lib.
Public portfolio	Next.js on Vercel	Stage-mode demo UI. Free hobby tier.
Hosting (private core)	Home server + Cloudflare Tunnel	\$0 hosting; Railway/Fly ~\$5 as fallback.
Secrets	.env + SOPS/age · Fernet for tokens	Encrypted at rest. Free.
Observability	Langfuse (self-host) or LangSmith	Trace every agent step. Free self-host / free tier.

Models, routing, agents & MCP tools

Model routing — cheapest capable model per task

A router (LiteLLM + a small policy) sends each task to the cheapest model that can do it well. Local models handle high-volume, low-stakes work for free; a hosted budget model handles reasoning; a premium model is reserved for the few outputs a human will actually read or send.

TIER	MODEL	USED FOR	COST
Local / bulk	Qwen2.5 7B/14B, Llama 3.1 8B (Ollama)	JD parsing, tagging, dedup, first-draft snippets, classification	Free
Fast / free-hosted	Groq Llama 3.3 70B · OpenRouter free models	Fit scoring, summarization, bulk drafting when local is busy	Free tier
Reasoning / budget	Claude Haiku · Gemini Flash · DeepSeek	Multi-step planning, idea synthesis, tricky matching	~\$0.25–1 / M tok
Premium / final	Claude Sonnet	Cover letters, founder pitches, anything you send to a human	Used sparingly
Embeddings	nomic-embed / bge-small (Ollama)	Semantic memory, job & contact similarity, dedup	Free (local)

Agent roster (LangGraph nodes)

A **Supervisor** routes each request or scheduled run to specialist agents. Agents share memory and the tool layer; each is a node (or subgraph) with a tight prompt and a small set of allowed tools.

JOBS Job Scout · JD Parser · Fit Scorer · Resume Tailor · Cover-Letter Writer · Connection Finder · Outreach Writer · Applier (executor) · Logger	STARTUP ENGINE Company Scout · Gap/Idea Synthesizer · Mini-Spec Writer · MVP Planner · Founder-Pitch Writer
NETWORK & FREELANCE Inner-Circle Curator · Touchpoint Writer · Freelance Prospector · Audit/Proposal Writer	RIDES & LIFE-OS Rides Watcher · Morning Briefing · Story Weaver · What-If Simulator · Learning Coach · Memory Manager

MCP tool servers

Capabilities are exposed as MCP servers so any model/agent can call them uniformly. Mix of off-the-shelf and a few custom ones.

- **Postgres MCP** — read/write Jarvis's own database (jobs, applications, contacts...).
 - **Browser MCP** (Playwright / browser-use) — open pages, fill forms, "check this website," scrape sandboxed.
 - **Web-search MCP** — Tavily or Serper for research (free tiers).
 - **Gmail + Calendar MCP** — read/draft email, read schedule (OAuth, least-privilege scopes).
 - **Filesystem MCP** — resume templates, generated PDFs, logs.
 - **Custom "Jobs" MCP** — wraps Adzuna, USAJobs, Greenhouse/Lever/Ashby feeds behind one clean tool.
 - **Custom "WhatsApp" MCP** — send/receive via the Baileys rides bot.
-

04 · DATA SOURCES & ACQUISITION

Exactly where every piece of data comes from

The whole system is only as good as its inputs. Prefer official APIs and public JSON feeds (green), fall back to authorized scraping (amber), avoid ToS-violating scraping (red) unless human-driven.

NEED	SOURCE	ACCESS	RISK
Jobs	Greenhouse / Lever / Ashby boards	Public JSON API per company	SAFE
	Adzuna	Official API, free tier	SAFE
	USAJobs (gov)	Official API, free key	SAFE
	Remotive · RemoteOK · Arbeitnow	Free public APIs	SAFE
	HN "Who is Hiring"	Algolia HN API, free	SAFE
	LinkedIn / Indeed listings	Scraping (no open API)	AVOID
Company research	Company site, blog, careers, docs	Polite scrape of public pages	OK
	Product Hunt · HN launches	Official / Algolia APIs	SAFE
	GitHub (repos, activity)	Official REST/GraphQL API	SAFE
People / contacts	LinkedIn profiles & messaging	Human-in-the-loop (see §09)	RISKY
	Your own Gmail / Calendar	Google OAuth (your consent)	SAFE
Rides	Your WhatsApp "Rides" group	Baileys (your session)	OK*
Freelance leads	Google Maps/Places, Upwork RSS, your contacts	APIs + manual	OK

*WhatsApp automation via unofficial libraries carries a small account-ban risk; keep it low-volume and human-like. The official WhatsApp Cloud API is the safer long-term path.

05 · JOB APPLICATION PACKAGE — MVP

The first thing we build, end to end

Goal: each day, find ~50 relevant roles in that day's theme, score fit, tailor a resume and cover letter, find people to reach out to, and deliver it as an approvable digest in Telegram. Auto-apply comes later, behind a gate.

DAILY PIPELINE

- 1 **Theme select.** Scheduler reads the weekly pattern — Day 1 mid-size, Day 2 university/research, Day 3 gov/public, Day 4 MNC/big-tech, Day 5+ custom — and sets today's target category.
- 2 **Scout.** Job Scout queries Adzuna + USAJobs + a curated list of Greenhouse/Lever/Ashby boards for that category, normalizes, dedups against past jobs by embedding similarity.
- 3 **Parse & score.** Local model parses each JD; Fit Scorer compares to your profile and returns Strong / Medium / Weak + a one-line reason.
- 4 **Tailor.** Resume Tailor picks the right variant (backend / data / AI / agents) and rewrites the summary + bullets for keyword match. Cover-Letter Writer drafts a "why me" paragraph.
- 5 **Connections.** Connection Finder identifies likely contacts (recruiter/eng/alumni); Outreach Writer drafts referral / info-chat / show-project messages. Sending is human-gated (§09).
- 6 **Deliver & log.** A Telegram digest lists ranked roles with resume + letter + drafts and Approve / Skip buttons. Every choice is logged and feeds preference learning.

DESIGN DECISION • EXECUTION MODE

Ship **semi-auto first** (Jarvis proposes, you approve in one tap). Add **mostly-auto** only after the drafts are consistently good — auto-apply runs through browser-use with per-run caps, a skip-list, and a kill switch. The "~50/day" target is a *discovery* goal; actual submissions stay quality-gated.

Fit rating output

STRONG — matches core skills + preferences

MEDIUM — partial match, worth a tailored shot

WEAK — skip unless you want a stretch

05•A • THE AGENT

Meet Sara Job — your autonomous application agent

Sara Job is the named face of the Job Package — a persistent AI agent (a LangGraph subgraph) who shows up every day and works the whole loop end to end: pick a job, read the JD, tailor your resume, apply on the company's career page, find the right people, draft connection notes and

emails, then document everything. She runs on the model router (local models for bulk, Claude for anything a human reads).

SJ

Sara Job · Application Agent

Runs daily · works one job at a time, fully · human-gated on submit · documents every action with proof

Sara's daily loop — per job

- 1 Pick.** Takes the next role from today's themed queue; skips duplicates, ghost posts, and reposts.
- 2 Read the JD.** Extracts requirements, keywords, seniority, location/visa constraints, and the screening questions the form will ask.
- 3 Fit & decide.** Scores Strong / Medium / Weak against your profile; below threshold she skips with a logged reason.
- 4 Tailor the resume.** Picks the right variant, rewrites summary + bullets to match JD keywords (real facts only), renders an ATS-clean PDF, and computes a keyword-match score.
- 5 Apply on the career page.** Drives the ATS (Greenhouse / Lever / Ashby / Workday) with browser-use — fills fields from your Profile Brain, uploads the resume, answers screening questions from memory. Stops at CAPTCHA / 2FA and hands off to you.
- 6 Capture proof.** Screenshots the confirmation and parses the confirmation email, so every application has evidence.
- 7 Find people.** Identifies a recruiter, the likely hiring manager, and 1–2 engineers or alumni via safe sources (careers/team pages, GitHub, public directories).
- 8 Draft outreach.** Writes a personalized LinkedIn connection note and a cold email (referral / info-chat / show-project variants). You send with one tap (LinkedIn stays human — \$09).
- 9 Document & schedule.** Writes a full application record + evidence to the log, adds it to the daily report, and schedules a follow-up.

Sara's brain — what powers every decision

Profile Brain

One structured fact base — contact, work authorization, locations, salary, availability, links, EEO answers — that fills every form consistently.

Screening-answer memory

Canned, learned answers to "why us / notice period / relocation" that improve as you edit them.

Resume variants

Backend / data / AI / agents master versions she tailors from — never fabricating, only reshaping real facts.

Preference memory

Your approvals/rejections retrain her fit scoring and outreach tone over time.

Suggested upgraded version — [my recommendations](#)

Beyond what you described, these modern touches make Sara noticeably smarter and safer:

- **ATS-score gate** — she won't submit below a keyword-match threshold; she fixes the resume or flags the role instead.
- **Proof-of-apply** — every submission stores a screenshot + parsed confirmation email as evidence.
- **Follow-up autopilot** — no reply in 5–7 days → she drafts a polite nudge automatically.
- **Reply radar** — watches Gmail for interview invites / rejections, updates the application status, and preps you.
- **Truth lock** — resume edits are constrained to real, verifiable facts; she never invents experience.
- **Daily standup** — a documented run report (Telegram + Markdown/Notion) with metrics: applied, skipped, drafted, follow-ups due.
- **Interview dossier** — when an interview lands, she assembles a company + role brief and likely questions.

GUARDRAILS

Sara only **rephrases and reorders real facts** — never fabricates. Every submit passes a human gate until you trust her on a given ATS; CAPTCHA, 2FA, and paywalled steps hand off to you. Per-site caps, a global kill switch, and a full audit trail apply.

New tables Sara needs

profile_facts

screening_answers

people

application_events

follow_ups

daily_reports

WHAT I NEED FROM YOU TO BUILD SARA

- All **resume variants** (PDF/DOCX) + a short master fact sheet.
- Exact **target titles + seniority + locations** and remote preference.
- **Work authorization / visa status**, salary range, availability, notice period.
- EEO / demographic answers you're okay auto-filling (or "leave blank").
- Companies to **prioritize or exclude**, and your preferred outreach tone.

05•B • DAILY INTAKE

Ask once each morning, then Sara runs

Your portal works in **two tiers**. The heavy facts are set *once* and reused forever; only the few things that change day-to-day are asked each morning. You answer one short message, and that answer kicks off Sara's run for the day.

TIER 1 • PROFILE BRAIN — SET ONCE

Name, contact, work authorization, EEO answers, resume variants, default locations, salary floor, tone. Changes rarely — you edit it only when something's actually different.

TIER 2 • DAILY INTAKE — ASKED EVERY MORNING

A pre-filled 20-second check-in: today's track, locations, salary, target count, mode, any focus/avoid companies, and a Start button. Defaults come from the Profile Brain, so most days you just tap **Start**.

The morning message (pre-filled, editable)

SJ

Sara Job
Good morning • Thu • Day 4 — MNC / big-tech theme

Here's today's plan — edit anything, then hit Start:

Track / resume	Data Engineer 
Locations	Remote + TX 
Salary floor	\$90k 

Target
~50 found · apply top fits

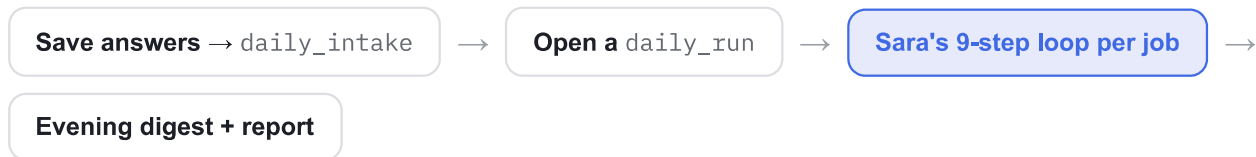
Mode
Semi-auto (approve each)

Focus / avoid
none

▶ Start today's run
Pause today

If you don't reply by a cutoff (e.g. noon), Sara can either wait, or auto-start with defaults — your choice, a Profile Brain setting.

What happens when you tap Start



New table

```

create table daily_intake (
  id          uuid primary key default gen_random_uuid(),
  run_date    date not null unique,
  track       text,                -- which resume variant / theme
  locations   text[],
  remote_ok   boolean,
  salary_floor int,
  target_count int default 50,
  mode        text,                -- semi_auto | auto
  focus_cos   text[],              -- companies to prioritize today
  avoid_cos   text[],              -- companies to skip today
  status      text default 'pending', -- pending|started|paused|done
  answered_at timestampz,
  created_at  timestampz default now()
);
  
```

06 • THE OTHER MODULES

Connections • Startups • Rides • Freelance • Life-OS

Each reuses the same agent + memory + database core. Built after the job MVP is stable.

Connection / Relationship Engine

Two levels: (a) per-application contacts tied to a job, and (b) an ongoing "inner circle" (friends, mentors, recruiters, founders). Tracks last-contact + context; each week surfaces 3–5 people due for a high-signal touchpoint with a ready draft. Storage: `contacts` + `touchpoints`.

Startup & Product Idea Engine

Scouts seed/Series-A companies (Product Hunt, HN, careers pages), finds gaps, and synthesizes small, buildable ideas with a mini-spec (problem → solution → architecture → mock screens). For ideas you like, it drafts an MVP plan and a tailored founder pitch: "I noticed X, I built this Y — demo + GitHub."

Rides Bot (already built)

Watches the WhatsApp "Rides" group, detects "need a ride" messages, DMs the asker for from/to/time/constraints, logs the request, and (optionally) matches riders. Wrapped as a WhatsApp MCP tool so its activity appears in the daily briefing.

Freelance Package

Finds/organizes leads (local businesses via Places API, online founders, existing contacts), analyzes each from public signals, proposes an automation project, and drafts an audit + scoped proposal + outreach. Tracks who responded and converted.

Life-OS Features

Morning Briefing (mission + priorities + micro-quest), Story-Mode weekly review, What-If Simulator for big decisions, Inner-Circle social engine, Learning Arena (daily card + XP), and Autopilot routines that log approvals/rejections to learn your preferences.

07 • DATABASES & SCHEMA

One Postgres, pgvector for memory

Core tables below. `memory` uses pgvector for semantic recall; large files (resume PDFs) live in object storage, referenced by URL.

companies	jobs
applications	resume_variants
contacts	outreach_messages
touchpoints	ideas / idea_specs
freelance_leads	rides
daily_runs	action_log (audit)
memory (vector)	config / preferences

CORE DDL — THE JOB LOOP

```

create extension if not exists vector;

create table companies (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  category    text,          -- midsize | university | gov | mnc | startup
  domain      text,
  ats         text,          -- greenhouse | lever | ashby | other
  research    jsonb,         -- scraped signals, pain points
  created_at  timestamptz default now()
);

create table jobs (
  id          uuid primary key default gen_random_uuid(),
  company_id  uuid references companies(id),
  source      text not null,  -- adzuna | usajobs | greenhouse | ...
  external_id text,
  title       text not null,
  location    text,
  remote      boolean,
  url         text,
  jd_text     text,
  jd_parsed   jsonb,         -- skills, seniority, keywords
  embedding   vector(768),   -- for dedup + similarity
  theme_day   text,          -- which daily category it belongs to
  fit_rating  text,          -- strong | medium | weak
  fit_reason  text,
  status      text default 'new', -- new|queued|applied|skipped
  seen_at     timestamptz default now(),
  unique (source, external_id)
);

create table applications (
  id          uuid primary key default gen_random_uuid(),
  job_id      uuid references jobs(id),
  resume_id   uuid references resume_variants(id),
  cover_letter text,
  mode        text,          -- semi_auto | auto
  submitted_via text,        -- portal | email | linkedin
  status      text default 'draft', -- draft|approved|submitted|rejected
  approved_by_human boolean default false,
  submitted_at timestamptz,

```



```
created_at    timestampz default now()  
);
```

Remaining tables (contacts, ideas, freelance_leads, rides, action_log, memory) follow the same shape — full DDL generated per module as it's built.

08 • API KEYS & ACCOUNTS REGISTRY

Everything you must sign up for

All secrets go in an encrypted `.env` (never in git). OAuth tokens are Fernet-encrypted in the DB. Nearly all of these are free.

SERVICE	WHAT / SCOPE	KEY	COST
Telegram	Bot interface	BOT_TOKEN	Free
Anthropic	Claude Haiku/Sonnet	ANTHROPIC_API_KEY	Pay/use
OpenRouter	Multi-model gateway (+free models)	OPENROUTER_API_KEY	Pay/use
Groq	Fast free Llama inference	GROQ_API_KEY	Free tier
Ollama	Local models + embeddings	– (local)	Free
Supabase	Postgres + pgvector + storage	SUPABASE_URL / _KEY	Free tier
Upstash	Redis queue/cache	UPSTASH_REDIS_URL	Free tier
Adzuna	Job search API	ADZUNA_APP_ID / _KEY	Free tier
USAJobs	Gov jobs API	USAJOBS_API_KEY	Free
Google Cloud	Gmail + Calendar OAuth (readonly + compose)	GOOGLE_OAUTH_*	Free
Google Places	Freelance local-business leads	GOOGLE_PLACES_KEY	Free credit
Tavily / Serper	Web search for research	TAVILY_API_KEY	Free tier
GitHub	Company/repo research	GITHUB_TOKEN	Free
WhatsApp (Baileys)	Rides bot session (QR pair)	– (session file)	Free
Vercel · Cloudflare	Public site + private tunnel	– (dashboards)	Free tier

09 · LINKEDIN & AUTOMATION RISK

The recommended safe path

REALITY CHECK

LinkedIn's User Agreement **prohibits automated scraping, bots, and auto-messaging**. They actively detect it and permanently ban accounts. There is no official API for job scraping or cold outreach for individuals. For your *only* professional account, an automated approach is not worth the ban risk.

Recommendation — "Jarvis drafts, you click":

- Jarvis **never logs into or scrapes LinkedIn automatically**.
- It finds connection *candidates* from safe sources (company careers pages, GitHub, public "team" pages, conference/alumni lists) and **drafts** personalized messages.
- Drafts land in your Telegram digest. You open LinkedIn yourself and paste/send — the sensitive click stays human. Fully ToS-compliant, zero ban risk.
- Optional power-up: a browser *extension* or bookmarklet that pre-fills the message box in your own logged-in session when you choose to — still you-initiated, not a background bot.
- For contact enrichment, use only compliant sources; do not resell or store scraped LinkedIn data.

If you later want higher volume, the compliant route is LinkedIn's official partner APIs (approval-gated, business-oriented) — not third-party scrapers.

10 • SCRAPING STRATEGY

Scrape politely, only where allowed

Scraping is the fallback, not the default. When used, it's on public pages that permit it, at human pace.

- **Order of preference:** official API → public JSON feed (Greenhouse/Lever/Ashby) → RSS → polite HTML scrape → (never) ToS-violating scrape.
 - **Respect** `robots.txt` and rate limits; identify with a real User-Agent; cache aggressively to avoid re-fetching.
 - **Tooling:** `httpx` + `selectolax` for static pages; Playwright + **browser-use** (AI navigates) for JS-heavy pages and form-filling.
 - **Sandbox:** the public demo's "check this website" runs Playwright in an isolated container with no access to private data or credentials.
 - **Auto-apply** uses browser-use on the employer's own ATS (Workday/Greenhouse) in *your* session — gated, capped, logged, kill-switchable.
-

11 • SECURITY & PRIVACY LAYERS

Guarding your accounts and data

Secrets & tokens

.env encrypted with SOPS/age; OAuth tokens
Fernet-encrypted in DB; no secrets in git or logs;
rotate on leak.

Least privilege

Gmail/Calendar scopes limited to compose/read
needed; each agent gets only the MCP tools it
needs.

Human approval gate

Any outbound action (apply, email, DM) requires
an explicit approve, plus a global kill switch and
per-run caps.

Audit log

action_log records every tool call + agent
decision for review and rollback.

Public/private isolation

Demo runs in a separate deployment with no real
credentials, mock tools, and synthetic data only.

Public-demo safety

Per-visitor rate limiting, PII redaction on all outputs,
sandboxed browser, prompt-injection guards on
"check this site."

Built for an unemployed student

ITEM	PLAN	MONTHLY
Compute (private core)	Home server + Cloudflare Tunnel	\$0
Database + storage	Supabase free tier	\$0
Redis	Upstash free tier	\$0
Bulk LLM + embeddings	Ollama local + Groq/free OpenRouter	\$0
Premium LLM (finals)	Claude Haiku/Sonnet, sparse use	\$5–12
Job / research APIs	Adzuna, USAJobs, Tavily free tiers	\$0
Public site	Vercel hobby	\$0
Domain	Amortized ~\$12/yr	~\$1
Total	Realistic solo run	≈ \$6–18 / mo

If you have no home server, swap in Railway/Fly (~\$5) — total still under ~\$25/mo. Cost scales mainly with how much premium-model output you approve.

13 · PHASED BUILD ROADMAP

Ship in small, working slices

PHASE 0 Week 1	✓ Foundation Repo, .env, Supabase schema, Telegram echo bot, LiteLLM router, Ollama running, one MCP (Postgres) wired.
PHASE 1 Week 2–4	✓ Job MVP (semi-auto) Scout → parse → fit → tailor resume + cover letter → daily Telegram digest with Approve/Skip. No auto-apply yet. This is the demo-able milestone.
PHASE 2 Month 2	✓ Connections + memory Contact tracking, weekly touchpoint suggestions, LinkedIn drafts (you-send), pgvector memory + preference learning.

PHASE 3

Month 3

✓ Auto-apply + startup engine

Gated browser-use auto-apply with caps/kill-switch; company scouting → idea synthesis → founder pitches.

PHASE 4

Month 4

✓ Freelance • rides • Life-OS

Integrate the rides bot, freelance prospecting, morning briefing, story review, what-if, learning arena.

PHASE 5

Month 5

✓ Public Jarvis demo

Portfolio site: synthetic personas, observatory log view, "Simulate Mock Monday," narrator personality.

14 • PUBLIC JARVIS (STAGE MODE)

The portfolio demo

A safe mirror on your portfolio that shows recruiters how the real system works — without touching private data.

- **Synthetic personas** (Job Hunter / Founder / Café Owner / Student) demo each module with mock data.
 - **Narrator personality:** "In Akhil's private Jarvis this would find 50 real jobs and auto-apply; here are 3 examples with dummy data."
 - **Observatory / log view:** live Thinking → Routing → Actions → Generation, which agent ran, which mock tools, fun (fake) metrics.
 - **"Simulate Mock Monday"** button walks a full day: briefing → job search → startup scan → rides request → freelance idea → story recap.
 - **Safety:** rate limiting, PII redaction, sandboxed browser — all covered in §11.
-

15 • OPEN QUESTIONS & NEXT STEPS

Decisions to confirm before Phase 0

- Do you have a **home server / always-on machine**, or should the private core run on Railway/Fly (\$5)?
- What are your **target roles + locations + resume variants**? (Needed to configure the Fit Scorer.)
- Confirm the **weekly day-theme pattern** and any hard filters (visa sponsorship, remote-only, salary floor).

- OK to **defer auto-apply** to Phase 3 and run semi-auto first? (Strongly recommended.)
- Which **Claude tier** do you want as the "final output" model given budget?

IMMEDIATE NEXT STEP

Answer the five questions above, then we start **Phase 0**: scaffold the repo, stand up Supabase + the Telegram bot, and wire the model router. From there, Phase 1 gets you a working daily job digest in ~2–4 weeks.